

团队协作指南

这份指南写给所有队友，包括第一次用 Git 和 GitHub 协作的人。按顺序照做就不会出错。看不懂或者不确定时，不要硬操作，先在群里问。

先弄懂几个词

词	大白话解释
仓库 (repo)	存放项目所有文件和修改历史的地方，我们的仓库在 GitHub 上
克隆 (clone)	把 GitHub 上的仓库完整复制一份到你自己的电脑
分支 (branch)	项目的一条"平行副本"。你在自己的分支上随便改，不会影响别人
main 分支	主分支，永远保持"随时能用"的状态，所有人共享
提交 (commit)	给你的修改存一个"进度存档"，附一句话说明改了些什么
推送 (push)	把你电脑上的存档上传到 GitHub
Pull Request (PR)	向团队申请"把我分支上的修改合并进 main"，队友检查通过后才真正合入

一条铁规则

永远不要直接修改 main 分支。所有改动都走：自己的分支 → 推送 → Pull Request → 队友检查 → 合并。

这样即使你改错了，也只是你自己分支的事，main 永远是好的。

第一次加入（只做一次）

我们用 Collaborator（协作者）模式：所有人直接操作同一个主仓库。不需要 Fork（Fork 是给陌生人给开源项目提交代码用的，我们是一个团队，用不上）。如果你已经 Fork 过，删掉那个 Fork，按下面重新来。

第 1 步：接受邀请。项目负责人会在 GitHub 上邀请你，去邮箱或 GitHub 右上角的通知里点接受。

第 2 步：把仓库克隆到你电脑上。打开终端（Windows 队友建议装 WSL2，群里可以问怎么装），执行：

```
git clone <主仓库地址>      # 地址在群公告里，注意是主仓库，不是你的Fork
cd <仓库目录>                # 进入刚克隆下来的文件夹
```

第 3 步：装开发环境。你只需要装三样东西：Node.js 22 以上、Docker Desktop、Git。装好后在仓库目录里依次执行：

<code>corepack enable</code>	# 让电脑自动使用项目指定的pnpm版本，不用自己装
<code>pnpm install</code>	# 安装项目依赖，所有人装出来的完全一样
<code>cp .env.example .env</code>	# 复制配置模板，然后打开.env填入自己的API Key
<code>docker compose up -d</code>	# 启动本地数据库（在Docker里，不用自己装数据库）
<code>pnpm dev</code>	# 启动项目，浏览器打开提示的地址就能看到页面

注意：目前仓库里还只有文档，没有代码，第 3 步要等第一个工程骨架 PR 合并后才能执行。现阶段先把 `doc/` 里的文档读一遍。

为什么不用大家手动统一版本？因为环境都写在仓库文件里了（Node 版本、依赖版本、数据库版本），照上面命令执行，每个人得到的环境天然一致。

日常开发：每次都走这七步

不管任务大小，每次都是同样的循环。

第 1 步：认领任务。在 GitHub Issue 或群里说“这个我来”，确认清楚要做什么、做到什么程度算完成。别和队友同时改同一个东西。

第 2 步：更新你本地的 main。别人可能已经合入了新代码，先同步：

<code>git switch main</code>	# 切换到main分支
<code>git pull origin main</code>	# 把GitHub上最新的main拉下来

第 3 步：创建自己的工作分支：

<code>git switch -c feat/lesson-canvas</code>	# 创建并切换到新分支
---	-------------

分支名规则：小写英文加短横线，前缀表示类型——

- `feat/xxx` 新功能，如 `feat/lesson-canvas`
- `fix/xxx` 修 bug，如 `fix/chat-stream-timeout`
- `docs/xxx` 改文档，如 `docs/backend-architecture`

第 4 步：写代码，本地验证能跑。如果你的修改影响了产品行为、接口或数据结构，同时更新 `doc/` 里对应的文档。

第 5 步：提交并推送：

```
git status # 先看看自己改了哪  
些文件  
git add <文件名> # 把要提交的文件加  
进来  
git commit -m "feat: add lesson canvas" # 存档并写一句说明  
git push -u origin feat/lesson-canvas # 上传到GitHub
```

提交说明的格式是 `类型：做了什么`，类型有：`feat`（新功能）、`fix`（修复）、`docs`（文档）、`refactor`（重构）、`test`（测试）、`chore`（配置杂项）。一次提交只做一件事。不要写“update”、“修改一下”这种看不出内容的说明。

第 6 步：创建 Pull Request。推送后打开 GitHub 仓库页面，会看到黄色提示条“Compare & pull request”，点它（或者去 Pull requests 标签页点 New）。目标分支选 `main`，然后写清楚：

1. 做了什么；
2. 为什么要做；
3. 你是怎么验证它能用的；
4. 有没有截图、风险或没做完的部分；
5. 是否更新了相关文档。

第 7 步：等检查和 Review，然后合并。满足这些条件才能合：

- 自动检查 (CI) 全部通过——"在我电脑上能跑"不算数，以 CI 为准；
- 至少一位队友看过并批准；
- 讨论都解决了。

合并方式用 Squash and merge（页面上合并按钮旁的下拉选项），它会把你分支上的一堆提交压成一条，让 main 的历史干干净净。合并后回到第 2 步，开始下一个任务。

新队友第一次走流程，建议先领一个小任务（比如改文档里一个错别字），完整跑一遍这七步，确认权限和流程都通，再做真正的功能。

怎么避免和队友冲突

- 动手前先认领，别两个人同时大改同一个文件；
- 每天开工前先做第 2 步（更新 main）；
- PR 保持小而清楚，尽量 1~2 天内完成合并，拖得越久越容易冲突。

出问题了怎么办

忘了切分支，直接在 main 上改了？别慌，只要还没 push 就没事。执行 `git switch -c feat/xxx`，你的修改会跟着到新分支上，然后正常提交。

push 或 pull 时提示冲突 (conflict)？说明你和队友改了同一处。先在群里说一声，和改那部分的队友一起解决。看不懂的代码不要删。

命令报错看不懂？把完整报错截图发群里。不要凭感觉重试各种命令，尤其永远不要用 `git push -f`（强制推送），它会覆盖别人的工作。

改乱了想重来？别自己删文件夹重克隆，先问群里，大部分情况一条命令就能救回来。

这些东西绝对不能提交进仓库

- API Key、密码、Token、私钥；
- `.env` 文件（真实配置只放自己电脑上，仓库里只有 `.env.example` 模板）；
- 学生真实个人信息；
- 大文件：模型权重、构建产物、临时缓存；
- 没确认过版权授权的教材、图片、视频。

如果不小心把密钥提交了，立刻在群里说——光删掉文件没用，历史里还在，需要负责人处理并更换密钥。

紧急修复也不例外

线上出了急事，同样从 main 创建 `fix/xxx` 分支走 PR，只是让队友优先 Review。除非项目负责人明确同意，任何人任何情况都不直接推 main。